



Draw It or Lose It

Lou Carron

Table of Contents

CS 230 Project Software Design Template

Error! Bookmark not defined.

Table of Contents	2
Document Revision History	2
Executive Summary	2
Requirements	2
Design Constraints	3
System Architecture View	3
Domain Model	3
Evaluation	4
Recommendations	Error! Bookmark not defined.

Document Revision History

Version	Date	Author	Comments
2.0	06/07/26	Lou Carron	Completed Project Two platform evaluation and recommendations for The Gaming Room.
3.0	06/20/26	Lou Carron	Completed Project Three recommendations and refined the design document for final submission.

Instructions

This completed software design document evaluates operating platforms, client support, and development tools for expanding Draw It or Lose It beyond Android into a scalable, browser-based distributed application.

Executive Summary

The Gaming Room has requested an evaluation of how to expand Draw It or Lose It from its current Android-focused implementation to a web-based application that supports users across desktop and mobile platforms. The recommended solution is to use a server-hosted web application with a responsive browser-based client. This allows Windows, macOS, Linux, Android, and iOS users to access the game through standards-based web browsers while the core game logic, session management, image library, timing rules, and persistence remain centralized on the server.

The best production platform for the server side is Linux because it provides robust web-hosting support, low licensing costs, reliable security maintenance, broad cloud availability, and a mature ecosystem for Java-based applications, databases, containers, and load balancing. Windows Server is also capable but normally adds licensing cost and is most valuable when the organization already depends on Microsoft infrastructure. macOS is useful for development and Safari/iOS testing but is not the strongest production hosting option. Mobile devices should be treated as clients, not production servers.

Requirements

The client requires Draw It or Lose It to support multiple platforms while keeping the game easy to maintain and scalable. The application should support desktop users on Windows, macOS, and Linux, as well as mobile users on Android and iOS. The client also requires a web-based application that can scale to thousands of players, render game images steadily during timed rounds, support team-based guessing, and keep the back-end service available to all clients. The software must communicate between front-end clients and the back-end server over a network, preserve consistent game behavior across platforms, and protect user and game-session information.

From a technical standpoint, the application should use a centralized server for game state, image/library access, timing, team data, and persistence. The client should use responsive HTML, CSS, and JavaScript so it can run consistently in modern browsers across desktops, tablets, and phones. The team should also plan for cross-browser testing, database-backed storage, secure authentication, encrypted network traffic, and maintainable deployment practices.

Design Constraints

The primary design constraint is that Draw It or Lose It must operate in a distributed web environment where many different client devices communicate with one back-end application. This means the application cannot rely on device-specific behavior or local-only storage. Instead, the server must manage the authoritative game state and expose secure APIs that all supported clients can use. Network reliability is also a constraint because players may connect from different devices and locations; the design must handle latency, timeouts, lost connections, and reconnection without corrupting the game session.

Another important constraint is cross-platform compatibility. The client interface must work in modern desktop and mobile browsers, which requires responsive design, accessible controls, and testing on Chrome, Edge, Firefox, and Safari. Security is also a constraint because player information, game sessions, and server resources must be protected. The design should use HTTPS/TLS, input validation, role-based access where needed, secure password handling if accounts are added, and regular operating system and dependency patching. These constraints push the design toward a centralized, scalable, browser-based architecture rather than multiple independent native applications.

System Architecture View

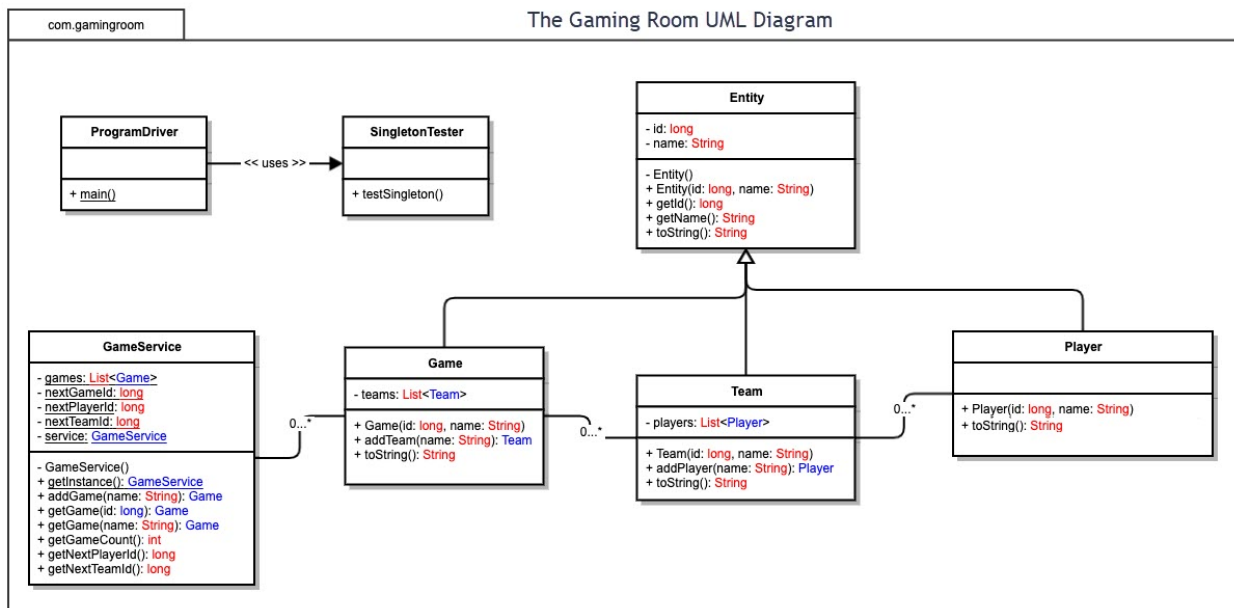
Draw It or Lose It should use a three-tier web architecture. The client tier will be a responsive browser-based interface used on Windows, macOS, Linux, Android, and iOS. The application tier will run on a hosted Linux server and expose secure web/API endpoints for game sessions, teams, guesses, timing, and image delivery. The data tier will store persistent game, team, player, and image metadata in a database, while static image assets can be stored in server file storage or cloud object storage. This structure separates user interface concerns from business logic and storage, making the system easier to scale, secure, and maintain.

Domain Model

The domain model represents the main objects needed to manage the Draw It or Lose It game structure. The Entity class is the parent class for Game, Team, and Player. Entity stores shared fields such as an identifier and name, which supports inheritance and reduces duplicate code. Game represents a game instance and contains a collection of Team objects. Team represents a team inside a game and contains a collection of Player objects. Player represents an individual participant. This object structure matches the business requirement that each game can have multiple teams and each team can have multiple players.

The GameService class manages the active game list and is implemented as a singleton. This supports the requirement that the application should have one shared service responsible for creating and locating games. The service assigns unique identifiers and prevents duplicate game names. The Game and Team classes also use collection searches before adding new teams or players, which helps maintain uniqueness inside their scope. The ProgramDriver and SingletonTester classes are used to test the application flow and confirm that the shared service instance works as expected.

The model demonstrates several object-oriented principles. Inheritance is used when Game, Team, and Player extend Entity. Encapsulation is used by keeping fields private and exposing controlled access through methods. The singleton pattern is used in GameService to control access to one shared service instance. The iterator pattern is used to walk through game, team, and player collections when checking for duplicates. These design choices support maintainability, code reuse, and consistency as the application expands into a distributed web application.



Evaluation

The following evaluation compares Mac, Linux, Windows, and Mobile Devices for server-side hosting, client-side support, and development tooling. The findings focus on the client's need for a scalable, browser-based, distributed game application.

Completed evaluation table:

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	macOS can host a web application for development, testing, or small internal use, but it is not the best production server platform for this project. Its strengths are stability, strong developer tooling, Unix-based command-line support, and excellent support for Safari and iOS testing. Its weaknesses are higher hardware cost, limited server-focused deployment options compared with Linux, and less common use in large-scale production web hosting. For The Gaming Room, macOS should be used mainly for development and compatibility testing, not as the primary host for thousands of game users.	Linux is the strongest server-side option. It is widely used for production web hosting, supports Java, web servers, databases, containers, scripting, and cloud deployment, and usually has low or no operating system licensing cost. Linux also scales well behind load balancers and supports automation through shell tools, Docker, Kubernetes, and cloud services. Its main weakness is that the team must have Linux administration and security expertise. For The Gaming Room, Linux provides the best balance of performance, reliability, cost control, and scalability.	Windows Server can host the application and is a strong option when the organization already uses Microsoft infrastructure, Active Directory, IIS, SQL Server, or Azure-centered administration. It offers mature management tools and enterprise support. The weakness is cost: Windows Server commonly involves server licensing and may require additional access licensing depending on the deployment model. It also has a larger licensing footprint than Linux for a cloud-scaled web application. Windows Server is capable, but it is less cost-effective than Linux for this use case.	Mobile devices should not be used to host the server-side application. Android and iOS devices are designed as clients with limited background execution, battery constraints, mobile network variability, and weaker operational controls than server operating systems. They are excellent platforms for players to access the game, but they should connect to a centralized back-end service hosted on Linux or Windows Server. Using mobile devices as servers would not meet the client need for reliability, security, and scalability.
Client Side	For macOS clients, the application should run in Safari, Chrome, and Firefox through a responsive web interface. Development cost is moderate because the team can use the same browser-based client code used for other platforms, but macOS-specific testing is still required because Safari can behave differently from Chromium-based browsers. The team needs front-end expertise in HTML, CSS, JavaScript, responsive design, and cross-browser testing. macOS is especially important for confirming compatibility with Safari and Apple device behavior.	For Linux clients, the application should run through modern browsers such as Chrome, Chromium, and Firefox. Cost is low because the same web client can support Linux without building a separate native desktop application. The main development consideration is testing browser behavior, fonts, input controls, and network communication on common Linux distributions. Linux client support should not require a separate team, but quality assurance should include Linux browser testing before release.	For Windows clients, the application should support Edge, Chrome, and Firefox. Windows is likely to be one of the largest desktop client environments, so testing should verify layout, performance, timers, image rendering, and user input in common Windows browsers. The cost and time impact are manageable because the same web codebase can support Windows. The team should still perform Windows-specific browser testing because browser versions, screen sizes, and hardware capabilities may vary widely among users.	For mobile clients, the application must support Android and iOS browsers using a responsive, touch-friendly interface. Mobile support increases testing time because the team must verify different screen sizes, touch controls, orientation changes, mobile network conditions, and performance on less powerful devices. The cost is still lower than building separate native Android and iOS applications because the first release can use one web client. However, the team needs mobile usability and responsive design expertise to make the game playable on phones and tablets.
Development Tools	Relevant tools include Visual Studio Code, IntelliJ IDEA, Git, Java/JDK tools, browser developer tools, Safari Web Inspector, and Xcode when iOS/Safari testing is required. macOS is valuable because it supports development for Apple platforms and allows direct Safari testing. The main licensing cost is Apple Developer Program membership if the team later distributes native iOS software; for the web client, most coding tools can be used at little or no cost.	Relevant tools include Java/JDK, Maven or Gradle, Git, Visual Studio Code or IntelliJ IDEA, Docker, command-line deployment tools, Nginx or Apache, and database tools such as MySQL or PostgreSQL clients. Linux is strong for server-side development because it closely matches the recommended production environment. Licensing cost can be low because many Linux distributions and development tools are open source, although enterprise support such as Ubuntu Pro or vendor support may add cost if the client wants formal support agreements.	Relevant tools include Visual Studio Code, IntelliJ IDEA, Eclipse, Java/JDK, Git, browser developer tools, Docker Desktop where appropriate, and Microsoft server or cloud tools if Windows Server or Azure is used. Windows is familiar to many development teams and supports full-stack web development well. Tooling costs vary: many tools are free, but Windows Server, some Microsoft enterprise tools, and some commercial IDE licenses may add cost depending on the chosen deployment and support model.	For mobile support, the primary web tools are responsive design frameworks, browser developer tools, device emulators, and real-device testing. Android Studio is useful for Android emulation and native Android development, while Xcode is needed for iOS simulation and native iOS workflows. If the project remains browser-based, the team can avoid separate native app builds at first. If native apps are added later, Kotlin/Java for Android and Swift for iOS would require more specialized mobile development expertise and additional publishing/testing effort.

Recommendations

The final recommendation is based on The Gaming Room's need to expand Draw It or Lose It into a distributed, browser-based application that supports multiple operating systems and mobile devices without duplicating the core application logic across each platform. The recommended design should be reliable enough for production, affordable to operate, secure by default, and flexible enough to scale as the number of players increases.

1. Operating Platform

The recommended operating platform for the server side of Draw It or Lose It is Linux, with Ubuntu Server LTS as a strong implementation choice. Linux is the best fit because it supports scalable web hosting, Java-based server applications, relational databases, containers, reverse proxies, load balancing, automation, and cloud deployment while avoiding the higher operating system licensing costs commonly associated with Windows Server deployments.

Linux also fits the client's goal of expanding to many computing environments because the server can expose one standards-based web application that desktop and mobile users' access through browsers. Windows, macOS, Linux, Android, and iOS clients do not need separate back-end implementations. Instead, the clients communicate with the centralized server through HTTPS and receive the same game rules, timing behavior, image content, team information, and session state.

This choice is also practical from an operations perspective. In a Linux virtual machine or homelab environment, the same core skills used in production can be practiced directly: installing packages, configuring services, checking logs, setting firewall rules, troubleshooting network connectivity, and managing server processes. That hands-on alignment makes Linux easier to validate, automate, and support as the application matures.

2. Operating System Architectures

Linux uses a layered and modular operating system architecture built around the Linux kernel. The kernel manages CPU scheduling, process execution, memory allocation, file systems, device drivers, networking, and security boundaries. Above the kernel are system services, user processes, libraries, shells, package managers, and application services. This design is effective for Draw It or Lose It because the game server can run as one or more isolated application processes while Linux manages resource allocation, networking, file access and process supervision.

The recommended server architecture should place the Java-based application behind a reverse proxy such as Nginx or Apache. The reverse proxy can terminate TLS, route traffic, serve static assets where appropriate, and forward application requests to the back-end service. The application service should manage game rules, session state coordination, team operations, and API responses. The database service should store persistent data such as users, games, teams, rounds, scores, and image references. This separation of responsibilities creates a clear production architecture rather than a single unmanaged process.

Linux also supports multiuser and multitasking operation, which are important for server reliability. Different services can run under separate users, limiting the impact of a compromise or application error. Multiple processes can run concurrently, allowing the web server, application server, database client, logging tools, monitoring agents, and maintenance tasks to operate at the same time. Because Linux is common in cloud and container environments, the same architecture can later be moved into containers or orchestrated across multiple servers if The Gaming Room grows.

3. Storage Management

The appropriate storage management system for the recommended platform is a relational database such as PostgreSQL running on Linux or provided as a managed cloud database. Draw It or Lose It needs to store structured, related data, including users, games, teams, players, rounds, guesses, timing data, scores, and references to image assets. A relational database is appropriate because it supports relationships, constraints, transactions, indexing, backups, and consistent queries across shared game data.

For the operating system file system, Linux should use a reliable journaling file system such as ext4 or XFS. Journaling helps protect file-system integrity after crashes or unexpected shutdowns. Application logs, configuration files, deployment files, and uploaded/static assets can be managed through standard Linux file permissions. Image assets should not be duplicated across clients; instead, they should be stored centrally in server storage or cloud object storage and referenced by the database. This keeps game data consistent and avoids version mismatches between devices.

The storage design should include scheduled database backups, tested restores, replication where appropriate, and separation between application files and persistent data. For production, managed database hosting or a dedicated database server would improve resilience because storage can be monitored, backed up, and scaled independently from the application tier. This approach is stronger than local client storage because the game state remains authoritative, consistent, and recoverable across every

supported platform.

4. Memory Management

Linux uses several memory management techniques that benefit the Draw It or Lose It server. Virtual memory allows each process to operate as if it has its own address space while the kernel maps that memory to physical RAM and, when necessary, swap space. Paging allows Linux to manage memory in fixed-size blocks and move less-used pages as needed. Process isolation prevents one process from directly corrupting another process's memory, which improves system stability and security.

Linux also uses caching aggressively. Frequently accessed files, database pages, and static assets can be held in memory to reduce disk access and improve performance. For a game such as Draw It or Lose It, this can help with image delivery, repeated reads of common game data, and faster response times during active sessions. Kernel-level resource controls and process management can also help prevent a single process from exhausting server resources.

The application itself must still be designed responsibly. The server should avoid storing unlimited game state in memory, release unused objects, limit large in-memory collections, and rely on the database or a controlled cache when data must survive restarts or be shared across multiple application instances. If the platform later uses containers; memory limits should be configured so that one application instance cannot starve the entire server. These techniques support stable gameplay even when many teams and players are active at the same time.

5. Distributed Systems and Networks

Draw It or Lose It should use a distributed client-server architecture. Desktop and mobile browsers act as clients, while the Linux-hosted back-end service manages the authoritative game state, game rules, timing, teams, guesses, scoring, and image delivery. Clients communicate with the server using HTTPS over TCP/IP networks. This design allows the same application to serve Windows, macOS, Linux, Android, and iOS users without creating separate native game engines for every platform.

As usage grows, the system can scale horizontally by placing multiple application servers behind a load

To support this, the application should keep API endpoints as stateless as practical and store persistent game state in a database or shared cache. Static assets can be delivered from centralized storage or a content delivery network. Monitoring should track server health, application errors, database performance, network latency, and unusual traffic patterns. This provides the operations team with early warning before small issues become outages.

The distributed design must also account for dependencies and failure points. If a player loses connectivity, the system should provide clear feedback and support session recovery where possible. If one application instance fails; the load balancer should route traffic to healthy instances. If the database becomes unavailable, the application should fail safely rather than corrupt game data. Timeouts, retries, health checks, logging, backups, and recovery procedures are all important because a distributed application depends on the network, servers, database, storage, and client browsers working together.

6. Security

Security must be built into the application, the Linux server, the network path, and the client experience. All communication between clients and the server should use HTTPS with TLS so that user information, game-session data, and authentication tokens are protected in transit. If user accounts are implemented, passwords must be salted and hashed using a modern password-hashing approach rather than stored in plain text. Sensitive database fields and backups should be encrypted where appropriate, especially if the application stores personally identifiable information.

Linux provides strong security capabilities for this environment. The server can use least-privilege service accounts, file permissions, firewalls, secure shell access controls, logging, package updates, and process isolation. Administrative access should be limited to authorized personnel, protected with strong authentication, and separated from normal application users. The application should validate input on every request, protect against injection attacks, restrict administrative operations through role-based access control and avoid exposing unnecessary error details to end users.

Security also requires operational discipline. The team should maintain a patching schedule for the operating system, application dependencies, runtime environment, database, and web server. Logs should be monitored for failed login attempts, suspicious request patterns, unexpected privilege changes, and application errors. Backups should be encrypted and tested. These controls protect user information on and between platforms and make the Linux-based recommendation stronger than a loosely managed server deployment.

Final Recommendation Summary

Ubuntu Server LTS on Linux is the strongest recommended platform for The Gaming Room. It provides the best overall combination of scalability, reliability, security, cost control, storage support, memory management, networking capabilities, and compatibility with modern distributed web application practices.

The client devices should remain browser-based clients, while the Linux server hosts the core application and coordinates shared game data. This architecture gives Draw It or Lose It a professional foundation for cross-platform expansion and long-term growth.

References

Canonical. Ubuntu Server and Ubuntu Pro documentation. Used to evaluate Linux server hosting, security maintenance, and enterprise support options.

PostgreSQL Global Development Group. PostgreSQL documentation. Used to evaluate relational database storage, consistency, backups, and transaction support.

The Linux Kernel Organization. Linux kernel documentation. Used to support discussion of process, filesystem, networking, and memory-management concepts.

Microsoft. Windows Server documentation and licensing information. Used to evaluate Windows Server as an alternative operating platform.

Apple Developer. Apple platform and developer documentation. Used to evaluate macOS/iOS development and testing considerations.

Android Developers. Android Studio and Android platform documentation. Used to evaluate Android/mobile development and testing considerations.

OWASP Foundation. Web application security guidance. Used to support recommendations for TLS, input validation, authentication, and secure design.